# RideFlow Complete Infrastructure and File Inventory

**Purpose:** Explain every major file, service, database table, secret, storage location, and operational process required to run RideFlow as a complete full-stack application.

> **The most important distinction:** The GitHub repository contains the application code, migrations, tests, and documentation. Your hosting account must provide the running server, database, authentication provider, object storage, notification transport, payment accounts, backups, monitoring, and domain configuration. A repository by itself is not a running RideFlow service.

## 1. The complete system at a glance

RideFlow is a React 19 frontend served by an Express server. tRPC procedures provide the typed application API. Drizzle ORM maps the TypeScript schema to a MySQL/TiDB-compatible database. S3-compatible object storage holds file bytes while the database stores metadata and authorization context. OIDC handles login. Stripe and Safaricom Daraja are external payment providers. The admin operations dashboard reads database-backed presence heartbeats and append-only activity events.

| System part | Runs where | Main responsibility |
| --- | --- | --- |
| Browser UI | User’s browser | Booking, driver mode, profile, admin views, preferences, payment-method selection |
| Express/tRPC server | Web-service host | Authentication callback, API procedures, fares, uploads, admin authorization, health endpoint |
| MySQL/TiDB database | Managed database host | Users, profiles, fare rules, quotes, ledger, presence, activity, file metadata |
| S3-compatible storage | Object-storage provider | Profile photos, driver documents, lost-item attachments, other private files |
| OIDC provider | Identity provider | Login, account identity, password, MFA, callback tokens |
| Stripe | Stripe account | Card payments, Connect marketplace setup, signed webhooks |
| Safaricom Daraja | Safaricom developer/merchant account | M-Pesa STK Push and asynchronous callbacks |
| Notification provider | HTTPS webhook or email service | Signup and operations notifications |
| GitHub | Private repository | Source versioning, collaboration, deployment trigger |
| Domain/DNS/TLS | Domain provider and host | Public URL, HTTPS, OIDC and payment callback reachability |
| Monitoring/backups | Host/provider or separate service | Logs, alerts, database recovery, incident evidence |

## 2. Repository files and what they do

### Frontend files

| Path | Purpose | Change when |
|---|---|---|
| `client/index.html` | Browser entry document and metadata | Branding, title, favicon, social metadata |
| `client/src/main.tsx` | React bootstrap, Query/tRPC providers, auth error handling | Provider or client bootstrap changes |
| `client/src/App.tsx` | Theme, routes, error boundary, top-level layout | Routes or global providers change |
| `client/src/pages/Home.tsx` | Main RideFlow workspace: customer, driver, profile, admin, booking, operations | Product screens or user flows change |
| `client/src/pages/NotFound.tsx` | Fallback route | Not-found experience changes |
| `client/src/pages/ComponentShowcase.tsx` | Component reference/testing page | UI component review changes |

| Path | Purpose | Change when |
| --- | --- | --- |
| `client/src/index.css` | Design tokens, layout, responsive rules, motion, payment selector and operations styles | Visual system or responsive behavior changes |
| `client/src/const.ts` | Browser login redirect helper and shared client constants | OIDC browser flow changes |
| `client/src/lib/trpc.ts` | Typed client binding | tRPC client configuration changes |
| `client/src/contexts/ThemeContext.tsx` | Light/dark theme state | Theme behavior changes |
| `client/src/_core/hooks/useAuth.ts` | Authentication state, login/logout, current user | Auth UI behavior changes |
| `client/src/components/ErrorBoundary.tsx` | Catches rendering failures | Global error display changes |
| `client/src/components/DashboardLayout.tsx` | Reusable dashboard shell | Internal dashboard layout changes |
| `client/src/components/Map.tsx` | Map integration boundary | Maps, routes, places, or directions are added |
| `client/src/components/ui/*` | Reusable Radix/shadcn-style UI primitives | Shared controls need changes |

`Home.tsx` is currently the main feature surface. As RideFlow grows, split large areas into dedicated pages and components rather than adding every feature to one file. Keep customer, driver, admin, and dispatch UI separate so authorization assumptions remain visible.

### Server and shared files

| Path | Purpose | Change when |
| --- | --- | --- |
| `server/_core/index.ts` | Production/development server entry, Express setup, routes, static serving, port binding | Host, health, middleware, or server lifecycle changes |
| `server/_core/env.ts` | Environment-variable access and validation | A secret or provider setting is added |
| `server/_core/context.ts` | Builds tRPC request context and current user | Authentication context changes |
| `server/_core/trpc.ts` | Public, protected, and admin procedure foundations | Authorization rules change |
| `server/_core/oauth.ts` | Provider-neutral OIDC/OAuth callback logic | Identity provider changes |
| `server/_core/cookies.ts` | Secure session-cookie behavior | Cookie or session policy changes |
| `server/_core/notification.ts` | Notification adapter boundary | Signup or operations notification provider changes |
| `server/_core/vite.ts` | Development/static asset integration | Development server behavior changes |
| `server/db.ts` | Database connection and domain helpers | Tables, queries, activity, presence, fare, or ownership logic changes |
| `server/routers.ts` | tRPC API contracts consumed by the frontend | Any user-facing server action changes |
| `server/fare.ts` | Server-side KSh fare calculation and commission | Pricing logic changes |

| Path | Purpose | Change when |
| --- | --- | --- |
| | split | |
| `server/payments.ts` | Stripe and Daraja configuration, readiness, callbacks, provider contracts | Payment providers or webhook behavior changes |
| `server/storage.ts` | S3-compatible storage upload and signed URL adapter | Bucket/provider changes |
| `server/index.ts` | Compatibility/server entry if retained by the project | Server startup compatibility changes |
| `shared/const.ts` | Shared constants and auth error values | Client/server contracts change |
| `shared/types.ts` | Shared application types | Cross-layer types change |

## Database and migration files

| Path | Purpose |
| --- | --- |
| `drizzle/schema.ts` | Source of truth for MySQL table definitions and inferred types |
| `drizzle/relations.ts` | Drizzle relation definitions |
| `drizzle/*.sql` | Generated migration history; apply in order to a new database |
| `drizzle/meta/*` | Drizzle migration snapshots and journal |
| `drizzle.config.ts` | Database and migration configuration |

Do not edit generated SQL casually. Update `drizzle/schema.ts`, generate a migration, inspect it, apply it to a disposable database, back up production, then apply it to production.

### Tests and documentation

| Path | Purpose |
| --- | --- |
| `server/*.test.ts` | Authorization, fare, storage, payment, operations, and sign-in regression tests |
| `docs/deployment-environment.template` | Names of required production environment variables without real secrets |
| `docs/self-hosting-configuration.md` | Provider-neutral self-hosting decisions |
| `docs/rideflow-free-hosting-and-database-guide.pdf` | Free/low-cost provider setup |
| `docs/rideflow-remaining-three-steps-guide.pdf` | Browser, payments, realtime, retention launch steps |
| `docs/rideflow-ubuntu-developer-guide.pdf` | Ubuntu developer setup and maintenance |
| `docs/rideflow-deployment-payments-guide.md` | Deployment and payment reference |
| `docs/rideflow-new-database-guide.md` | New database and migration reference |
| `docs/rideflow-complete-code-guide.md` | Code map and maintenance handbook |
| `todo.md` | Project history and completion checklist |
| `README.md` | Repository quick-start and operational orientation |

## 3. Database tables required for the current application

The current schema is defined in `drizzle/schema.ts` and is applied through the numbered migration files.

| Table | Stores | Sensitive? | Important controls |
| --- | --- | --- | --- |
| `users` | Auth identity, email, role, sign-in timestamps | Yes | Unique identity, admin role, secure session mapping |
| `rideflow_profiles` | Customer/driver role, phone, vehicle, insurance, verification state | Yes | Authenticated access; driver review status |
| `rideflow_files` | File metadata and private storage references | Yes | Store bytes in S3, not database; signed URLs; MIME/size validation |
| `rideflow_admin_settings` | Owner email, notification email, admin configuration | Yes | Admin-only writes; verified ownership transfer |
| `rideflow_fare_rules` | City pricing, minimum, safety fee, commission basis points | No/Business-sensitive | Admin-only writes; server-side quote calculation |

| Table | Stores | Sensitive? | Important controls |
| --- | --- | --- | --- |
| `rideflow_fare_quotes` | Route labels, distance/time, fare components, KSh totals, status | Business-sensitive | Server-created; expiry; immutable financial history expectations |
| `rideflow_ledger_entries` | Rider charge, driver earning, platform commission, refund, tip, payout | Financial | Append-only application behavior; reconcile provider IDs separately |
| `rideflow_presence` | Current user status, current view, last heartbeat | Personal/operational | Protected admin read; stale heartbeat becomes offline |
| `rideflow_activity_events` | Sign-ins, quotes, uploads, admin changes, summaries, metadata | Security/operational | Append-only; retention policy; avoid secrets and raw GPS |

The 5% commission is represented by `platformCommissionBps` with a default of 500 basis points. The server computes the platform amount and driver amount; the browser must never be the authority for a payable total.

## What is not yet a complete production rides database

The current schema is the foundation, not the entire Uber-scale operating model. A production dispatch system still needs explicit trip, driver-location, driver-availability, assignment, cancellation, payment-transaction, refund, dispute, payout, message, rating, favorite-driver, scheduled-ride, trip-share, safety-contact, and audit entities. Add these through schema-first migrations and tests before using real riders and drivers.

## 4. Server capabilities required in production

The web server must provide an HTTPS public URL and bind to the host and port supplied by the provider. It must expose the health endpoint, serve the built frontend, accept `/api/trpc` requests, handle the OIDC callback, and receive Stripe/Daraja callbacks.

The minimum server process needs access to the database, JWT/session secret, OIDC credentials, S3 credentials, notification credentials, and sandbox payment credentials. These values belong in the provider's secret manager. They must not appear in `client/src`, committed `.env` files, GitHub Actions logs, screenshots, or PDFs.

For GPS dispatch, add a realtime transport or managed realtime service. Keep the main API authoritative for trip state. Authenticate every realtime connection, authorize every trip channel, limit location frequency, handle reconnects, and store the latest location separately from the generic activity feed.

## 5. Storage and file lifecycle

RideFlow's database stores metadata such as original filename, MIME type, size, purpose, storage key, review state, and user ID. The object-storage provider stores the actual bytes. Create separate prefixes or buckets for profile photos, driver documents, and lost-item attachments if your provider supports it. Keep driver documents private and return short-lived signed URLs only to authorized users or reviewers.

The web host's local filesystem is temporary on most free server tiers. Never use it for uploaded files, SQLite, generated reports, or database backups. Store backups in a different provider or offline encrypted storage.

## 6. Complete environment-variable inventory

Use `docs/deployment-environment.template` as the source for names. The following categories must be populated in the deployment secret manager.

| Category | Variables |
| --- | --- |
| Runtime | `NODE_ENV`, `PORT`, `PUBLIC_APP_URL` |
| Database | `DATABASE_URL` |
| Sessions | `JWT_SECRET` |
| OIDC | `OAUTH_ISSUER_URL`, `OAUTH_TOKEN_URL`, `OAUTH_USERINFO_URL`, `OAUTH_CLIENT_ID`, `OAUTH_CLIENT_SECRET`, `VITE_OAUTH_AUTHORIZE_URL`, `VITE_OAUTH_CLIENT_ID`, `VITE_OAUTH_SCOPE` |
| Storage | `S3_BUCKET`, `S3_REGION`, `S3_ENDPOINT`, `S3_ACCESS_KEY_ID`, `S3_SECRET_ACCESS_KEY`, `S3_FORCE_PATH_STYLE`, `S3_PUBLIC_BASE_URL`, `S3_SIGNED_URL_TTL_SECONDS`, `S3_SERVER_SIDE_ENCRYPTION` |
| Notifications | `RIDEFLOW_ADMIN_EMAIL`, `NOTIFICATION_PROVIDER_URL`, `NOTIFICATION_PROVIDER_TOKEN` |
| Stripe | `PAYMENTS_STRIPE_ENABLED`, `STRIPE_SECRET_KEY`, `VITE_STRIPE_PUBLISHABLE_KEY`, `STRIPE_WEBHOOK_SECRET`, `STRIPE_CONNECT_CLIENT_ID` |
| Daraja | `PAYMENTS_DARAJA_ENABLED`, `DARAJA_ENVIRONMENT`, `DARAJA_CONSUMER_KEY`, `DARAJA_CONSUMER_SECRET`, `DARAJA_SHORTCODE`, `DARAJA_PASSKEY`, `DARAJA_INITIATOR_NAME`, `DARAJA_SECURITY_CREDENTIAL`, `DARAJA_CALLBACK_BASE_URL` |

Use sandbox values first. When moving to production, change provider mode deliberately, rotate keys, update callback URLs, and test the new environment independently.

# 7. Deployment order from empty infrastructure

## Phase A: Accounts and source

Create the private GitHub repository, hosting account, database account, S3-compatible storage account, OIDC application, Stripe account, and Daraja account. Enable multi-factor authentication on owner and provider accounts. Keep provider recovery codes in a secure password manager.

### Phase B: Database

Create the MySQL-compatible database, create a least-privilege application user, configure TLS, set `DATABASE_URL`, run migrations in order, and verify the expected tables. Create a backup before changing the schema.

### Phase C: Storage

Create the private bucket, restrict public access, create an application access key with only the required bucket permissions, configure S3 variables, and test one non-sensitive image upload.

### Phase D: Web server

Connect GitHub, create a Node web service, set the build command to `corepack enable && pnpm install --frozen-lockfile && pnpm build`, set the start command to `pnpm start`, add secrets, deploy, inspect logs, and test `/healthz`.

### Phase E: Authentication

Register the exact deployed callback URL with the OIDC provider, configure issuer/token/userinfo/authorize variables, sign in with a test account, verify a session cookie, and test logout. Create the initial admin only through the documented verified-user process.

### Phase F: Payments and notifications

Configure Stripe test mode and Daraja sandbox callbacks. Test successful, failed, duplicate, invalid-signature, and timeout cases. Verify that ledger effects are idempotent. Configure an HTTPS notification provider and test signup/admin notifications without SMTP assumptions.

### Phase G: Operations

Enable logs and alerts, verify admin operations, test presence heartbeats, define activity retention, back up the database, test restoration, and document who can rotate each secret. Do not invite real riders until dispatch, safety, support, disputes, and payment reconciliation are ready.

## 8. Ubuntu operations commands

```
cd /home/ubuntu/rideflow
pnpm install --frozen-lockfile
pnpm check
pnpm test
pnpm build
pnpm dev
```

For a schema change:

```
# 1. edit drizzle/schema.ts
pnpm drizzle-kit generate
# 2. inspect the new drizzle/*.sql file
# 3. apply it to a disposable database first
pnpm drizzle-kit migrate
# 4. back up production, then apply the approved migration
```

For a release:

```
git status
git add <reviewed-files>
git commit -m "Describe the change"
git push origin main
```

Use the hosting dashboard to redeploy, inspect logs, and roll back to the last known-good commit. Do not use destructive database commands without a verified backup and rollback plan.

## 9. Backups, monitoring, and recovery

Back up the database, storage metadata, and provider configuration separately. A database backup does not include S3 file bytes, and an S3 backup does not include database authorization metadata. Keep at least one encrypted copy outside the primary provider.

Monitor HTTP health, server errors, database connection failures, storage upload failures, payment callback failures, webhook signature failures, stale presence writes, and retention-job failures.

Maintain a written incident procedure with the owner, technical operator, payment contact, database contact, and identity-provider contact.

The recovery test is complete only when a new server can be connected to a restored database and storage bucket, an administrator can log in, a test quote can be created, a test upload can be retrieved securely, and a sandbox callback can be reconciled.

## 10. What must be added before a real public launch

The current repository is a strong portable foundation, but a real ride-sharing operation also needs production trip and dispatch tables, realtime driver location, matching and reassignment, customer/driver chat, safety escalation, refunds and disputes, payout reconciliation, ratings policy, scheduled rides, trip sharing authorization, support tooling, privacy controls, retention enforcement, rate limiting, monitoring, and legal review.

Free hosting is appropriate for learning and private testing. Before public launch, move to services with reliable backups, support, monitoring, a documented SLA, and enough capacity for realtime traffic and payment callbacks.

## References


[1]: https://render.com/docs/web-services Render, “Web Services.”

[2]: https://aiven.io/docs/products/mysql/concepts/mysql-free-tier Aiven, “Aiven for MySQL free tier.”

[3]: https://supabase.com/pricing Supabase, “Pricing & Fees.”

[4]: https://docs.stripe.com/webhooks Stripe, “Receive Stripe events in your webhook endpoint.”

[5]: https://developer.safaricom.co.ke/apis/MpesaExpressSimulate Safaricom Daraja, “M-Pesa Express Simulate.”